



**JEMORE**  
TOGETHER FOR SUCCESS

# Formazione Backend Pratica



# Database

## Overview

*E' un insieme organizzato di dati memorizzati in modo strutturato ed accessibili.  
Un database è composto da:*

- *Database Management System ( DBMS )*
- *Linguaggio di interrogazione*
- *Modello dei dati*

*I principali 2 tipi di database sono: relazionali e non-relazionali.*



# Supabase

*Supabase è una piattaforma Open Source che fornisce la possibilità di costruire un backend completo basato su PostgreSQL.*

*Molto utile per avere un modo facile e veloce per gestire i dati e utenti nel nostro sito.*



# Pool di connessione

Come stabiliamo una connessione con il db?

**Dobbiamo stabilire una connessione tra il database e il nostro sito, come facciamo?**

**Per fare ciò utilizziamo una stringa di connessione, la chiave per accedere ai dati!**

**Il pool gestisce le connessioni già aperte permettendoci di riutilizzarle in modo da non aprirne sempre di nuove quando interagiamo con il sito.**

```
import { Pool } from "pg";

declare global {
  var _postgresPool: Pool | undefined
}

class PostgresDB {
  private static instance: PostgresDB
  public pool: Pool

  private constructor() {
    if (!process.env.DATABASE_URL) {
      throw new Error('POSTGRES_URL environment variable is not defined')
    }

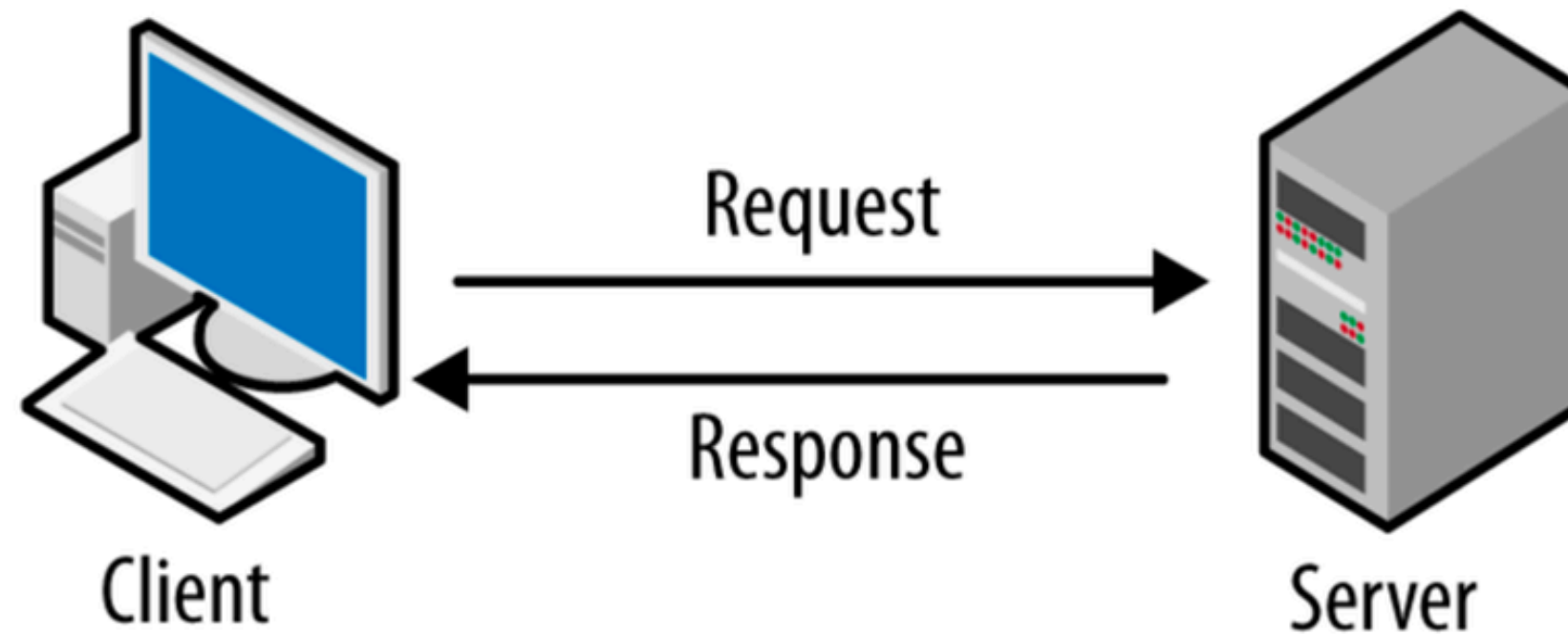
    // Configurazione con connection string
    this.pool = global._postgresPool || new Pool({
      connectionString: process.env.DATABASE_URL,
      max: 20, // numero massimo di client nel pool
      idleTimeoutMillis: 30000,
      connectionTimeoutMillis: 2000,
      ssl: process.env.NODE_ENV === 'production' ? { rejectUnauthorized: false } : false
    })

    // In sviluppo, mantieni il pool in globalThis
    if (process.env.NODE_ENV === 'development' && !global._postgresPool) {
      global._postgresPool = this.pool
    }

    this.pool.on('error', (err) => {
      console.error('Unexpected error on idle PostgreSQL client', err)
    })
  }
}
```

**Esempio di Pool con stringa di connessione:**

# Modello client-server



 **FRONTEND**

**BACKEND**



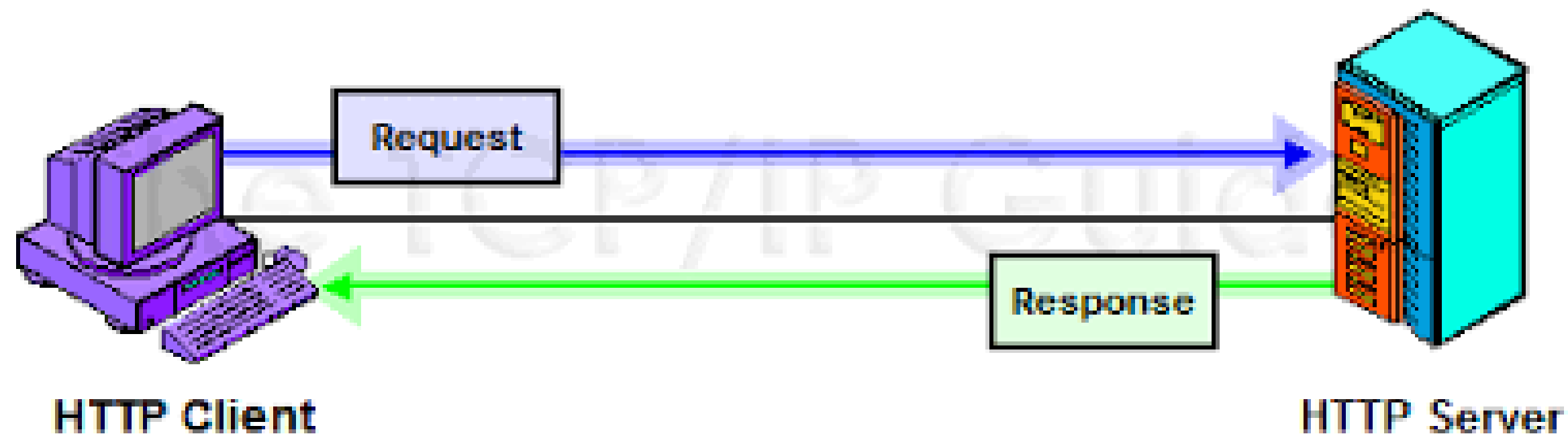


# Protocollo HTTP

Hypertext Transfer Protocol

E' un protocollo di rete applicativo usato per la trasmissione di informazioni su architetture di tipo client-server.

Il client esegue delle RICHIESTE e il server fornisce delle RISPOSTE, entrambe di tipo HTTP.



# Richieste HTTP

## Metodi

- **GET:** richiesta di risorse.
- **POST:** creazione di risorse.
- **PUT:** aggiornamento di risorse.
- **DELETE:** eliminazione di una risorse.
- **HEAD:** per ricevere informazioni su una risorsa.
- **OPTIONS:** per ricevere informazioni riguardante funzionalità del server e i formati supportati.
- **TRACE:** per ricevere richieste inviate al server ( per debug )
- **CONNECT:** per creare connessioni sicure.

**Come l'utente interagisce con il  
database tramite il sito?**



# Fetch

Come l'utente interagisce con il database

**Quando un utente interagisce con il sito questo può inviare e ricevere dati dal database, questo scambio è gestito da delle funzioni chiamate fetch.**

**Utente clicca  
Bottone:**

**Sito Web:  
usa la funzione fetch**

**Database:  
Fornisce la risposta con i  
dati**

```
// GET ALL
export async function fetchPosts() {
  try {
    const res = await fetch("/api/blog") // get a blog

    if (!res.ok) { //se va male...
      console.error("Errore durante la fetch");
    }

    const data = await res.json(); //salvo i dati sottoforma di json
    return data;
  } catch (error) {
    console.error("Errore nella fetch:", error);
    return [];
  }
}
```

**Esempio di funzione fetch**

# Richieste HTTP: GET

Esempio di Query GET per prendere tutti gli articoli

```
// Funzione per ottenere tutti i post
export async function GET() {
  try {
    const result = await db.query("SELECT * FROM posts");
    return NextResponse.json(result.rows, { status: 200 });
  } catch (error) {
    console.error("Error fetching posts:", error);
    return NextResponse.json({ error: "Failed to fetch posts" }, { status: 500 });
  }
}
```

- Funzione asincrona
- Interrogazione del database
- Controllo di consistenza
- Restituzione dei dati o errore



# Richieste HTTP: POST

Esempio di Query POST per aggiungere un articolo

```
// Funzione per aggiungere un post
export async function POST(request: Request) {
  try {
    const body = await request.json();

    // Validazione dei dati
    if (!body.title || !body.content) {
      return NextResponse.json(
        { error: "Title and content are required" },
        { status: 400 }
      );
    }

    // Aggiungi il nuovo post
    const result = await db.query(
      "INSERT INTO posts (title, content) VALUES ($1, $2) RETURNING *",
      [body.title, body.content]
    );

    // Salva il post in un file o in un database (simulato qui con un array)
    return NextResponse.json({ message: "Post added successfully", post: result.rows[0] }, { status: 201 });
  } catch (error) {
    return NextResponse.json({ error: "Failed to add post" }, { status: 500 });
  }
}
```

- **Funzione asincrona**
- **Prende dei dati in input dall'utente**
- **Controllo di consistenza**
- **Restituisce un codice, che rappresenta com'è andata l'operazione:**
  - **201, tutto ok**
  - **500, errore di qualche tipo**

# Route dinamiche

*Next.js permette l'implementazione di route dinamiche, utili per situazioni in cui vogliamo creare più pagine (con contenuti diversi) usando un solo file di codice*

*Il contenuto di questa route cambia in base al suo URL, tutto grazie ad un parametro dinamico.*

*Nel nostro caso ci permette di strutturare delle pagine specifiche per ogni articolo con URL che variano in relazione all'id dell'articolo stesso.*

*Per riconoscere una route dinamica si utilizzano le parentesi quadre [ ]*

```
bash
└─ /pages
   ├── index.js
   ├── about.js
   └── [id].js ← 🧩 dinamico
```

```
GET http://localhost:3000/api/blog/6
```

# Richieste HTTP: PUT

Esempio di Query PUT per modificare un dato articolo

```
// Funzione per modificare un post
export async function PUT(request: Request, context: { params: { id: string } }) {
  try {
    const params = await context.params;
    const id = Number(params.id);
    const body = await request.json();

    // Validazione dei dati
    if (!id || !body.title || !body.content) {
      return NextResponse.json(
        { error: "ID, title, and content are required" },
        { status: 400 }
      );
    }

    // Aggiorna il post
    const result = await db.query(
      "UPDATE posts SET title = $1, content = $2 WHERE id = $3 RETURNING *",
      [body.title, body.content, id]
    );
    if (result.rowCount === 0) {
      return NextResponse.json({ error: "Post not found" }, { status: 400 });
    }

    return NextResponse.json(
      { message: "Post updated successfully", post: result.rows[0] },
      { status: 200 }
    );
  } catch (error) {
    return NextResponse.json({ error: "Failed to update post" }, { status: 500 });
  }
}
```

- Molto simile alla POST
- Funzione asincrona
- Prende dei dati in input dall'utente
- Necessita anche dell'ID dell'elemento da modificare
- Restituisce un codice, che rappresenta com'è andata l'operazione:
  - 201, tutto ok
  - 500, errore di qualche tipo



# Richieste HTTP: DELETE

Esempio di Query DELETE per rimuovere un articolo

```
// Funzione per eliminare un post
export async function DELETE(request: Request, context: { params: { id: string } }) {
  try {
    const params = await context.params;
    const id = Number(params.id);

    // Validazione dell'ID
    if (!id) {
      return NextResponse.json(
        { error: "ID is required" },
        { status: 400 }
      );
    }

    // Elimina il post
    const result = await db.query("DELETE FROM posts WHERE id = $1 RETURNING *", [id]);

    if (result.rowCount === 0) {
      return NextResponse.json({ error: "Post not found" }, { status: 404 });
    }

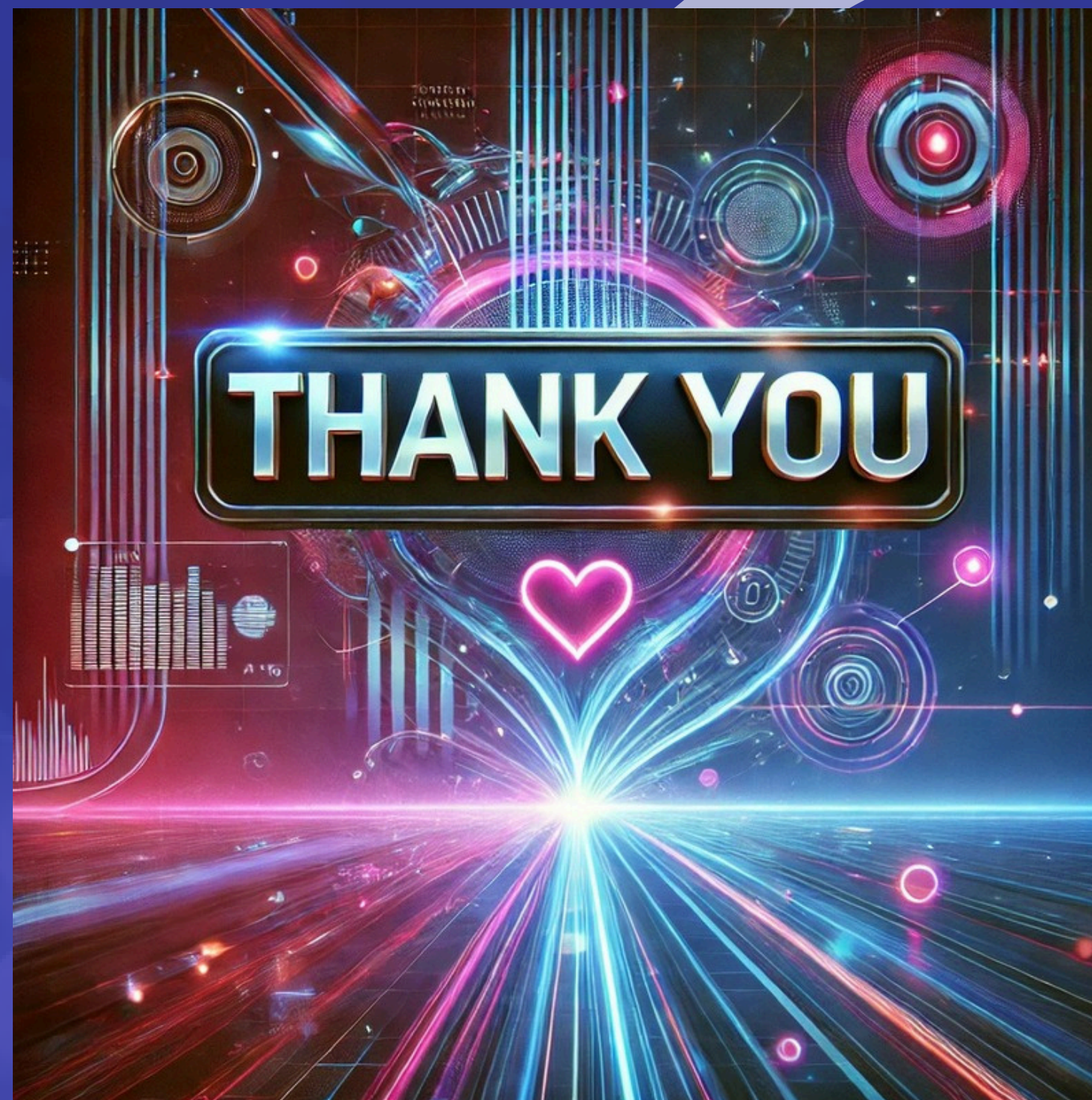
    return NextResponse.json(
      { message: "Post deleted successfully", post: result.rows[0] },
      { status: 200 }
    );
  } catch (error) {
    return NextResponse.json({ error: "Failed to delete post" }, { status: 500 });
  }
}
```

- Funzione asincrona
- Necessita dell'ID dell'elemento da eliminare
- Restituisce un codice, che rappresenta com'è andata l'operazione:
  - 201, tutto ok
  - 500, errore di qualche tipo





**JEMORE**  
TOGETHER FOR SUCCESS



[www.jemore.it](http://www.jemore.it)  
E SIA LODATA FOGGIA